

Metody Inteligencji Obliczeniowej

Sprawozdanie 8



Wydział Fizyki i Informatyki Stosowanej

Mikhail Shupliakou

29.05.2026

Spis treści

1	Wstęp	3
2	Zadanie 1	3
2.1	Cel zadania	3
2.2	Proces rozwiązania	3
2.3	Wyniki eksperymentów i wnioski	4
2.4	Analiza wyników i ocena wariantów	5
3	Zadanie 2	6
3.1	Cel drugiego zadania	6
3.2	Opis realizacji drugiego zadania	6
3.3	Wyniki eksperymentów i wnioski z porównania algorytmów	8

Algorytm roju cząstek (PSO - particle swarm optimization)

1 Wstęp

Algorytm roju cząstek (ang. *Particle Swarm Optimization*, PSO) jest jedną z najpopularniejszych metaheurystyk inspirowanych naturą, opracowaną w 1995 roku przez Jamesa Kennedy’ego oraz Russella Eberharta. Algorytm ten bazuje na symulacji zachowań społecznych obserwowanych w stadach ptaków lub ławicach ryb, które współpracują ze sobą w celu znalezienia optymalnego miejsca w przestrzeni (np. źródła pożywienia).

W algorytmie PSO populacja potencjalnych rozwiązań traktowana jest jako rój, a każdy jej pojedynczy element jest cząstką przemieszczającą się w wielowymiarowej przestrzeni poszukiwań. Każda cząstka posiada własną pozycję oraz wektor prędkości, które są dynamicznie modyfikowane w kolejnych iteracjach (epokach). Ruch cząstek opiera się na trzech głównych czynnikach: sile bezwładności, komponencie poznawczym (pamięć o własnym najlepszym osiągniętym wyniku) oraz komponencie społecznym (wiedza o najlepszej pozycji znalezionej przez jakikolwiek element roju). Dzięki prostej strukturze matematycznej i braku konieczności obliczania gradientu funkcji celu, algorytm PSO jest niezwykle efektywnym narzędziem do rozwiązywania złożonych problemów optymalizacyjnych.

2 Zadanie 1

2.1 Cel zadania

Celem niniejszego zadania jest implementacja oraz analiza wydajności algorytmu roju cząstek (ang. *Particle Swarm Optimization*, PSO) w problemie poszukiwania globalnego minimum złożonej funkcji nieliniowej dwóch zmiennych na zadanym przedziale. Badanie opiera się na empirycznej ocenie wpływu kluczowych parametrów sterujących algorytmu — współczynnika bezwładności (w) oraz współczynników uczenia dla komponentu poznawczego (c_1) i społecznego (c_2) — na szybkość zbieżności, stabilność wyników (mierzoną odchyleniem standardowym serii niezależnych uruchomień) oraz jakość końcowego rozwiązania, co ostatecznie pozwala na wyłonienie optymalnej konfiguracji dla rozpatrywanego problemu.

2.2 Proces rozwiązania

Proces realizacji zadania podzielono na trzy główne etapy: definicję badanej funkcji celu, implementację algorytmu optymalizacji PSO oraz przygotowanie środowiska do eksperymentów numerycznych.

W pierwszej kolejności zaimplementowano analizowaną funkcję matematyczną z wykorzystaniem biblioteki `numpy`. Rozwiązanie to pozwala na wydajną i wektoryzowaną ocenę przystosowania (ang. *fitness*) poszczególnych cząstek w badanej przestrzeni poszukiwań:

```
def fitness_function(X):
    x, y = X[0], X[1]
    term1 = 2 * np.log(np.abs(x + 0.2) + 0.002)
    term2 = np.log(np.abs(y + 0.1) + 0.001)
    term3 = np.cos(3 * x)
    term4 = 2 * np.sin(3 * x * y)**2
    term5 = np.sin(y)**2
    term6 = - x**2 - 0.5 * y**2
    return term1 + term2 + term3 + term4 + term5 + term6
```

Następnie opracowano główny mechanizm algorytmu roju cząstek. Po wstępnej, losowej inicjalizacji pozycji oraz prędkości cząstek wewnątrz dopuszczalnego przedziału $[-1, 1]$, uruchamiana jest główna pętla iteracyjna. Kluczowym elementem algorytmu jest matematyczny model przemieszczania się roju. W każdej epoce aktualizowany jest wektor prędkości cząstek na podstawie ich własnych dotychczasowych doświadczeń (komponent poznawczy sterowany parametrem c_1) oraz najlepszego wyniku całego roju (komponent społeczny z parametrem c_2). Zmiany te są dodatkowo modyfikowane przez współczynnik bezwładności w . Pozycje cząstek są na bieżąco korygowane z wykorzystaniem mechanizmu obcinania (ang. *clipping*), aby nie przekraczały wyznaczonych granic:

```
# Aktualizacja prędkości i pozycji cząstek
V = w * V + c1 * r1 * (pbest_X - X) + c2 * r2 * (gbest_X - X)
X = X + V

# Ograniczenie wyjścia poza przedział [-1, 1]
for d in range(dim):
    X[:, d] = np.clip(X[:, d], bounds[d][0], bounds[d][1])
```

Ostatnim krokiem było zdefiniowanie scenariuszy badawczych. Przygotowano strukturę danych przechowującą wszystkie wymagane w zadaniu kombinacje parametrów c_1 , c_2 oraz w . Aby upewnić się, że uzyskane wyniki będą wiarygodne statystycznie i zniwelować wpływ losowej inicjalizacji roju, skrypt został zaprogramowany w taki sposób, aby dla każdej konfiguracji wykonać 10 niezależnych uruchomień algorytmu. W trakcie obliczeń gromadzone są dane niezbędne do późniejszego wyznaczenia średniej, odchylenia standardowego oraz wygenerowania wykresów zbieżności.

```
# Definicja wariantów parametrów do zbadania
variants = [
    {"name": "c1=0, c2=2, w=0.5", "c1": 0, "c2": 2, "w": 0.5},
    {"name": "c1=2, c2=0, w=0.5", "c1": 2, "c2": 0, "w": 0.5},
    {"name": "c1=1, c2=1, w=0.5", "c1": 1, "c2": 1, "w": 0.5},
    {"name": "c1=0.5, c2=1.5, w=0.5", "c1": 0.5, "c2": 1.5, "w": 0.5},
    {"name": "c1=2.2, c2=2.2, w=0.5", "c1": 2.2, "c2": 2.2, "w": 0.5},
    {"name": "c1=2, c2=2, w=0.2", "c1": 2, "c2": 2, "w": 0.2},
    {"name": "c1=2, c2=2, w=0.9", "c1": 2, "c2": 2, "w": 0.9}
]
```

2.3 Wyniki eksperymentów i wnioski

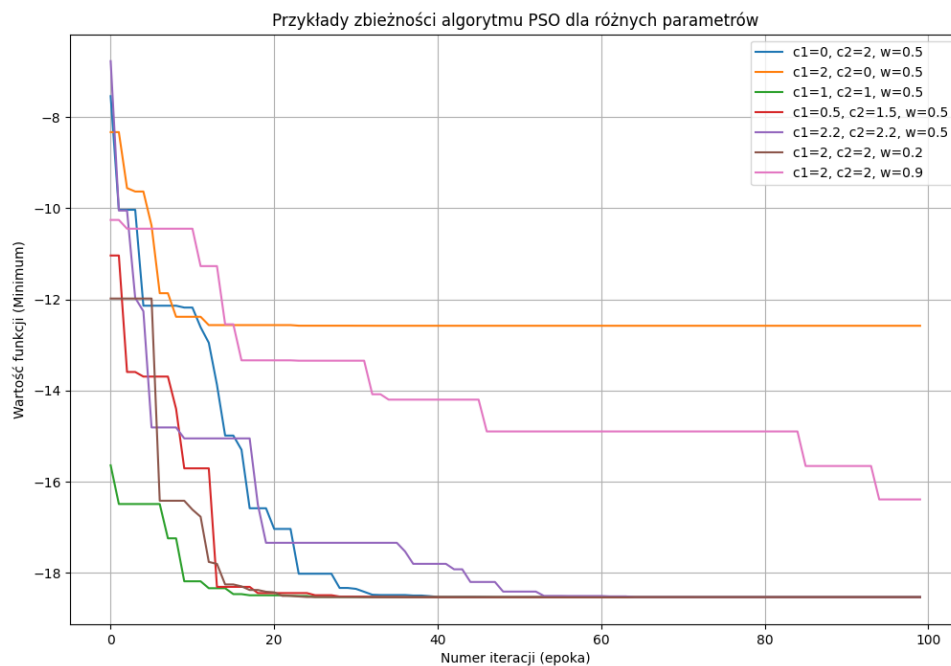
W celu oceny wpływu parametrów sterujących algorytmu PSO na proces poszukiwania minimum globalnego, przeprowadzono serię eksperymentów dla siedmiu różnych konfiguracji. Dla każdego wariantu wykonano 10 niezależnych uruchomień al-

gorytmu. Zestawienie uśrednionych wartości funkcji przystosowania oraz odchylenia standardowego zaprezentowano w Tabeli 1.

Tabela 1: Wyniki optymalizacji algorytmem PSO dla różnych konfiguracji parametrów (średnia z 10 uruchomień)

Wariant	c_1	c_2	w	Średnie minimum	Odchylenie stand.
Tylko społeczny	0	2	0.5	-18.539478	0.000000
Tylko poznawczy	2	0	0.5	-12.142353	1.474631
Zrównoważony	1	1	0.5	-18.539478	0.000000
Pośredni	0.5	1.5	0.5	-17.520383	3.057271
Wysokie przyspieszenie	2.2	2.2	0.5	-18.539185	0.000483
Niska bezwładność	2	2	0.2	-18.539478	0.000000
Wysoka bezwładność	2	2	0.9	-16.014501	0.806768

Dynamikę zbieżności algorytmu dla przykładowych uruchomień poszczególnych wariantów przedstawiono na Rysunku 1.



Rysunek 1: Przykładowe przebiegi zbieżności algorytmu PSO dla badanych konfiguracji parametrów.

2.4 Analiza wyników i ocena wariantów

Na podstawie uzyskanych wyników można sformułować następujące wnioski dotyczące wpływu poszczególnych parametrów na działanie algorytmu:

- **Brak komponentu społecznego ($c_1 = 2, c_2 = 0$):** Wariant ten osiągnął zdecydowanie najgorsze rezultaty (średnia: -12.14) oraz charakteryzował się wysokim odchyleniem standardowym. Potwierdza to teoretyczne założenia PSO – brak wymiany informacji o najlepszym globalnym rozwiązaniu (brak inteligencji).

cji roju) sprawia, że cząstki błędzą losowo, opierając się wyłącznie na własnym doświadczeniu, co uniemożliwia sprawną zbieżność.

- **Brak komponentu poznawczego** ($c_1 = 0, c_2 = 2$): Ku zaskoczeniu, wariant ten poradził sobie doskonale, osiągając globalne minimum (-18.539478) w każdej próbie (odchylenie 0.0). Może to wynikać ze specyfiki badanej funkcji – silne przyciąganie do lidera roju pozwoliło na błyskawiczną eksploatację obiecującego obszaru. W ogólnym przypadku takie ustawienie niesie jednak ogromne ryzyko przedwczesnej zbieżności (utknięcia w minimum lokalnym).
- **Wpływ współczynnika bezwładności (w)**: Porównanie wariantów dla $w = 0.2$ oraz $w = 0.9$ (przy $c_1 = c_2 = 2$) wyraźnie obrazuje mechanizm eksploracji i eksploatacji. Niska bezwładność ($w = 0.2$) pozwoliła cząstkom na szybkie wyhamowanie i precyzyjne "dostrojenie" się do minimum globalnego. Z kolei zbyt wysoka bezwładność ($w = 0.9$) nadała cząstkom zbyt duży pęd, przez co "przelatywały" one nad optimum, osiągając gorszy wynik końcowy i wykazując mniejszą stabilność.
- **Balans parametrów**: Konfiguracja zrównoważona ($c_1 = 1, c_2 = 1, w = 0.5$) zapewniła bezbłędną zbieżność oraz perfekcyjną stabilność (odchylenie równe zero). Równie dobrze sprawdził się wariant o niskiej bezwładności ($c_1 = 2, c_2 = 2, w = 0.2$). Warianty pośrednie ($c_1 = 0.5, c_2 = 1.5$) wykazały pewną niestabilność (najwyższe odchylenie standardowe: 3.05), co sugeruje, że w niektórych uruchomieniach algorytm mógł utknąć w suboptymalnych rejonach przestrzeni poszukiwań.

Podsumowanie: Z przeprowadzonych badań wynika, że do optymalizacji zadanej funkcji najlepiej sprawdził się **wariant zrównoważony** ($c_1 = 1, c_2 = 1, w = 0.5$) oraz **wariant z niską bezwładnością** ($c_1 = 2, c_2 = 2, w = 0.2$). Oba te warianty konsekwentnie, w każdym z 10 uruchomień, odnajdywały najgłębsze minimum funkcji (-18.539478), gwarantując stuprocentową powtarzalność wyników, co czyni je najbardziej niezawodnymi konfiguracjami dla rozpatrywanego problemu.

3 Zadanie 2

3.1 Cel drugiego zadania

Celem zadania jest porównanie algorytmu roju cząstek (PSO) z algorytmem genetycznym (GA) w procesie optymalizacji badanej funkcji. Analiza obejmuje zestawienie jakości znalezionej minimum oraz czasu działania obu metod przy założeniu identycznej liczby epok.

3.2 Opis realizacji drugiego zadania

Realizacja drugiego zadania polegała na zestawieniu wydajności algorytmu roju cząstek (PSO) z algorytmem genetycznym (GA), dostosowanym do optymalizacji w dziedzinie ciągłej. W tym celu konieczne było zaimplementowanie od podstaw algorytmu genetycznego oraz przygotowanie mechanizmu testującego, który zapewni

równe warunki ewaluacji dla obu metod (identyczna liczba epok oraz równy rozmiar populacji i roju).

W pierwszej kolejności zdefiniowano główną pętlę algorytmu genetycznego. W każdej iteracji oceniane jest przystosowanie osobników, a następnie stosowany jest mechanizm selekcji turniejowej. Do nowej populacji wybierany jest najlepszy osobnik z losowo dobranej trójki, co zapewnia odpowiednią presję selekcyjną:

```
# Selekcja turniejowa
new_population = np.zeros_like(population)
for i in range(pop_size):
    tournament = np.random.choice(pop_size, 3, replace=False)
    winner_idx = tournament[np.argmin(fitness[tournament])]
    new_population[i] = population[winner_idx]
```

Kolejnymi kluczowymi etapami operacji genetycznych są krzyżowanie oraz mutacja. Ze względu na ciągły charakter rozpatrywanego problemu (liczby rzeczywiste), zastosowano krzyżowanie arytmetyczne, które z zadaniem prawdopodobieństwem (70%) tworzy potomstwo będące liniową kombinacją genotypów rodziców. Następnie wprowadzono operator mutacji, polegający na dodaniu losowego szumu o rozkładzie jednostajnym do genotypu wybranych osobników (z prawdopodobieństwem 10%):

```
# Krzyżowanie arytmetyczne (crossover)
for i in range(0, pop_size - 1, 2):
    if np.random.rand() < 0.7:
        alpha = np.random.rand()
        parent1, parent2 = new_population[i].copy(), new_population[i+1].copy()
        new_population[i] = alpha * parent1 + (1 - alpha) * parent2
        new_population[i+1] = alpha * parent2 + (1 - alpha) * parent1

# Mutacja (dodanie losowego szumu)
for i in range(pop_size):
    if np.random.rand() < mutation_rate:
        mutation_val = np.random.uniform(-0.2, 0.2, dim)
        new_population[i] += mutation_val
```

Podobnie jak w przypadku algorytmu PSO, nałożono ograniczenia dziedziny (przedział od -1 do 1), używając funkcji obcinającej, aby nowo wygenerowane osobniki nie opuszczały dopuszczalnej przestrzeni poszukiwań.

Ostatnim krokiem było stworzenie środowiska porównawczego. Zaprogramowano pętlę testową, która uruchamia oba algorytmy wyznaczoną liczbę razy (10 niezależnych uruchomień) przy identycznych parametrach wejściowych (100 epok, populacja licząca 30 osobników). Do precyzyjnego pomiaru czasu działania poszczególnych przebiegów wykorzystano standardową bibliotekę `time`:

```
for _ in range(runs):
    # Test PSO
    start_time = time.time()
    best_pso = pso_compare(num_particles=population_size, max_iter=epochs)
    pso_times.append(time.time() - start_time)

    # Test GA
    start_time = time.time()
    best_ga = genetic_algorithm(pop_size=population_size, max_iter=epochs)
    ga_times.append(time.time() - start_time)
```

Zgromadzone w ten sposób dane posłużyły do późniejszego wyznaczenia uśrednionych wartości optymalizowanych minimów oraz średniego czasu wykonania dla obu badanych heurystyk.

3.3 Wyniki eksperymentów i wnioski z porównania algorytmów

W celu weryfikacji skuteczności algorytmu roju cząstek (PSO), jego wyniki zestawiono z klasycznym algorytmem genetycznym (GA). Oba algorytmy zostały uruchomione z identycznymi parametrami początkowymi: liczba epok wynosiła 100, a rozmiar populacji (lub roju) ustalono na 30 osobników. Wykonano 10 niezależnych uruchomień dla każdej z metod, a uśrednione wyniki dotyczące znalezionej minimum oraz czasu wykonania przedstawiono w Tabeli 2.

Tabela 2: Porównanie wydajności algorytmów PSO i GA (średnia z 10 uruchomień)

Algorytm	Średnie minimum	Odchylenie stand.	Średni czas wykonania [s]
PSO	-18.539478	0.000000	0.026640
GA	-14.991989	1.732998	0.086371

Analiza wyników i ocena wydajności

Na podstawie zebranych danych empirycznych można wyciągnąć następujące wnioski:

- **Jakość rozwiązania i stabilność:** Algorytm PSO wykazał się absolutną bezbłędnością w analizowanym problemie. W każdym z 10 uruchomień odnalazł to samo, optymalne minimum globalne (wartość -18.539478), co skutkuje zerowym odchyleniem standardowym. Z kolei algorytm genetyczny (GA) w tej samej liczbie epok osiągnął zauważalnie gorszy wynik średni (-14.991989) oraz charakteryzował się wysokim odchyleniem standardowym (1.732998). Wskazuje to na fakt, że GA często grzązł w minimach lokalnych i potrzebowałby znacznie większej populacji lub liczby iteracji, aby zbliżyć się do wyników PSO.
- **Czas działania algorytmu:** Algorytm roju cząstek okazał się ponad trzykrotnie szybszy w działaniu (średnio ~ 0.026 s w porównaniu do ~ 0.086 s dla GA). Różnica ta wynika z samej natury obu heurystyk. Aktualizacja pozycji w PSO opiera się na prostych i niezwykle szybkich operacjach wektorowych (dodawanie i mnożenie macierzy prędkości). Z kolei GA wymaga wykonywania w każdej epoce znacznie bardziej kosztownych obliczeniowo operacji, takich jak selekcja turniejowa (wymagająca sortowania lub wielokrotnego porównywania) oraz sprawdzanie warunków probabilistycznych dla krzyżowania i mutacji.

Podsumowanie: Zestawienie obu metod jednoznacznie faworyzuje algorytm roju cząstek dla badanego problemu optymalizacji funkcji ciągłej. W zadanych warunkach (100 epok, populacja 30) PSO okazał się narzędziem znacznie bardziej precyzyjnym, w 100% stabilnym oraz ponad trzykrotnie szybszym obliczeniowo niż klasyczny algorytm genetyczny.